# ■■ CyberShield: AI-Powered Network Intrusion Detection System

**The Beginner-to-Pro Master Guide: How It Works, Why It Works & Interview Secrets**

## 1. Networking 101: Simple Everyday Analogies

Before diving into code, let's understand how computer networks work using a simple apartment building analogy:

• **What is an IP Address?:** Think of an IP address like a home mailing address. Every computer connected to the internet has a unique IP (e.g. 192.168.1.100). When you browse a website, your computer sends letters from your IP to Google's IP.

• **What is a Port Number?:** Imagine an IP is an office building, and Ports are the different numbered doors inside. Door 80 & 443 are the front lobby for public websites. Door 22 is the secure basement server room. Door 53 is the office phone directory (DNS). There are 65,535 possible doors.

• **What is a Packet?:** Internet data cannot travel all at once. Big files are broken into tiny digital envelopes called 'Packets' (each with sender IP, destination IP, port, and a few hundred bytes of data).

• **What is an Intrusion Detection System (NIDS)?:** A NIDS is like an intelligent security camera and guard stationed at the front gate. It inspects every single packet passing through the network, watches for suspicious behavior, and sounds the alarm before attackers can break in.

## 2. The Cyber Attacks We Detect (In Simple Words)

### Port Scanning (The Doorknob Rattler):

An attacker wants to find an unlocked door into your computer. In a Port Scan, the attacker quickly tries doors 21, 22, 23, 80, 443, and 3389 in rapid succession. Our system notices when one person touches 8+ different doors in 8 seconds and sounds the alarm.

### Denial of Service - DoS (The Phone Jammer):

Imagine 1,000 angry callers spamming a pizza shop's phone line simultaneously so real customers get a busy tone. In a DoS attack, an attacker spams 50+ packets per second to crash the server. Our system measures packet velocity and catches floods immediately.

### Stealth Scans (NULL & XMAS Attacks):

TCP packets normally carry communication flags (like 'Hello' = SYN or 'Goodbye' = FIN). Attackers sometimes craft illegal packets with NO flags (NULL scan) or ALL flags lit up like a Christmas tree (XMAS scan) to confuse basic firewalls. Our engine inspects flag combinations and catches them.

### Zero-Day & Statistical Anomalies:

What if a hacker invents a brand new attack never seen before? Static rules will miss it. That is why we use Machine Learning (Isolation Forest) to catch any packet that doesn't look like normal, peaceful web traffic.

## 3. Machine Learning Deep-Dive: How Isolation Forest Works

**The Crowd Metaphor:** Imagine you are in a room with 1,000 corporate accountants wearing dark business suits. Suddenly, someone walks in wearing a neon rainbow clown costume. How many questions do you need to ask to spot the clown? *Just one: 'Who has a rainbow wig?'* You isolate them instantly! But if you wanted to isolate one specific accountant, you would have to ask 50 questions (height, hair color, tie pattern, shoe size).

**How the Algorithm Does This in Math:**

• **Normal Traffic** = The Accountants (HTTP web browsing, Google DNS queries). They look similar, so decision trees need many deep cuts to separate them.

• **Malicious Traffic** = The Clown (Huge packet size, strange ports, rapid burst rates). Decision trees isolate them with very few

splits near the root.
• When an anomaly is isolated quickly with few splits, the algorithm gives it a **negative outlier score**, triggering an AI anomaly alert!

## 4. The 12 Clues Extracted from Every Packet

Every packet that enters our system is converted into 12 numerical clues (features):

| Clue # | Feature Name | What It Tells the AI |
|---|---|---|
| 1 | packet_length | Byte size. Huge packets indicate data theft; tiny packets indicate ping scans. |
| 2, 3, 4 | proto_tcp, udp, icmp | Is it TCP (web/apps), UDP (video/DNS), or ICMP (network pings)? |
| 5 | is_well_known_port | 1 if standard public port (like 80 or 443), 0 if a weird high port. |
| 6 | dst_port_norm | Destination port number scaled between 0.0 and 1.0. |
| 7, 8, 9, 10 | SYN, ACK, FIN, RST | TCP handshake flags. SYN=connecting, ACK=talking, FIN=closing, RST=resetting. |
| 11 | packet_rate | Speed: How many packets this sender sent in the last 3 seconds (detects floods). |
| 12 | port_diversity | How many different port doors this sender rattled in 8 seconds (detects scans). |

## 5. Step-by-Step: What Happens When You Click 'Port Scan'?

- **Step 1 (User Action):** You click the 'Port Scan' button on the React dashboard in your browser.
- **Step 2 (API Call):** React sends an HTTP POST request to http://127.0.0.1:8000/api/simulate.
- **Step 3 (Traffic Injection):** traffic_simulator.py fires 16 rapid TCP SYN packets targeting ports 21, 22, 23, 80, 443, 3389.
- **Step 4 (Feature Extraction):** features.py measures the speed (>15 pps) and unique ports (12 unique doors) and builds the 12-number vector.
- **Step 5 (Dual Brain Inspection):**
  - *Heuristic Rule:* Sees 12 ports in 8s ➔ triggers 'Port Scanning Attempt' (+55 risk points).
  - *Isolation Forest:* Isolates the rapid SYN burst ➔ flags statistical anomaly (+35 risk points).
- **Step 6 (Risk Scoring):** Combines base + heuristic + ML points = **90.0 / 100 Risk Score (CRITICAL)**.
- **Step 7 (Database Storage):** alert_manager.py saves the alert to the SQLite database (nids.db).
- **Step 8 (Live WebSocket Push):** The backend broadcasts the alert over ws://localhost:8000/ws/traffic. React receives it and animates the row onto your screen instantly with zero page reload!
- **Step 9 (AI Copilot):** You click 'Explain with AI'. The AI Copilot tags it with **MITRE ATT&CK; T1046**, explains it in plain English, and writes a 1-click Windows PowerShell command to block the attacker.

## 6. Project Folders Explained in Plain English

| File / Folder | What It Does (Beginner Friendly) |
|---|---|
| backend/app/main.py | The server entrance. Runs FastAPI and keeps the live WebSocket pipe open. |
| backend/app/ml/detector.py | The chief security officer. Runs both the ML model and port scan/DoS rules. |
| backend/app/engine/ai_copilot.py | The AI incident assistant. Maps threats to MITRE ATT&CK; and writes firewall rules. |
| backend/app/models.py | The database blueprint. Defines how alerts and packet logs are stored in SQLite. |
| frontend/src/App.jsx | The visual command center (React). Renders the dark cyber UI, gauges, tables, and charts. |
| start.bat | 1-click magic starter. Double-click it to start both backend and frontend servers together. |

## 7. How to Start the Project Anytime

**1-Click Method:** Open folder C:\Users\shiva\OneDrive\Desktop\ai-nids and double-click **start.bat**.
**Browser Address:** Open Chrome or Edge and go to **http://localhost:5173**.

## 8. Resume Section & How to Explain in Interviews

### Copy-Paste for Resume:

**CyberShield: AI-Powered Network Intrusion Detection & Incident Response System**
*Python, FastAPI, Scikit-learn (Isolation Forest), React 19, Tailwind CSS, WebSockets, SQLAlchemy, Scapy*
• Engineered a real-time NIDS combining an unsupervised **Isolation Forest ML model** with stateful heuristic rules to detect port scans, volumetric DoS floods, and zero-day anomalies with sub-second latency.
• Built an asynchronous **FastAPI** backend streaming live telemetry over full-duplex **WebSockets** to a responsive **React 19** SOC dashboard with dynamic 0-100 risk scoring.
• Integrated an **AI Threat Copilot** mapping detections to the **MITRE ATT&CK; Framework** (T1046, T1498) and auto-generating 1-click Windows PowerShell and Linux IPtables firewall quarantine scripts.

### Top 3 Interview Questions & Answers:

**1. What is this project in one sentence?**
*Answer:* CyberShield is a real-time network defense system that uses unsupervised machine learning and heuristic rules to detect cyberattacks as packets travel across a network, visualize them on a live SOC dashboard, and automatically generate firewall containment scripts.

**2. Why did you use Isolation Forest instead of a standard classifier?**
*Answer:* Supervised models need pre-labelled attack data and fail on brand-new zero-day exploits. Normal web traffic represents 99.9% of flows. Isolation Forest is unsupervised: it isolates outliers at shallow decision tree depths based on structural variance without needing pre-existing attack signatures.

**3. What was the hardest engineering challenge?**
*Answer:* Coordinating asynchronous streaming with stateful detection. We needed to calculate rolling packet velocity (PPS) and unique destination port diversity across sliding time windows in real time, score each packet with ML, persist alerts to SQLite, and broadcast them over WebSockets to React with zero UI lag.